检索增强生成与上下文工程

RAG & Context Engineering

中文修订版 · 图表与公式随文排版

原作者：@techNmak · Understanding AI 系列 · https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Core	核心	
Context Engineering	上下文工程	
Retrieval-augmented Generation	检索增强生成	RAG
Hierarchical Retrieval	层次检索	
Continued Pretraining	继续预训练	
Contextual Retrieval	上下文检索	
Query Transformation	查询变换	
Prompt Engineering	提示词工程	
Semantic Chunking	语义分块	
Context Assembly	上下文组装	
Context Window	上下文窗口	
Chunk Overlap	块重叠	
Late Chunking	延迟分块	
Fine-tuning	微调	
Performance	性能	
Chunk Size	块大小	
Generation	生成	
Retrieval	检索	
Retriever	检索器	
Reranking	重排序	
Evidence	证据	
Pipeline	流水线	
Parsing	解析	
Query	查询	Q
Model	模型	
Multi-query Retrieval	多查询检索	
Parametric Knowledge	参数化知识	

英文原词	中文译名	缩写／符号
Context Compression	上下文压缩	
Ground Truth	基准真值	
Rank Fusion	排名融合	
Re-indexing	重新建立索引	
Provenance	来源溯源	
Citation	引用	
Latency	延迟	
Index	索引	
Embeddings	嵌入向量	
Chunking	分块	
Packing	打包	
Models	模型	
Hallucination	幻觉	
Inference	推理	
Generator	生成器	
Queries	查询	
Vector	向量	
Token	词元	
Benchmark	基准测试	
Reranker	重排序器	
Accuracy	准确性	
Embedding	嵌入	
Tokens	词元	
Recall	召回率	
Structure-aware Chunking	结构感知分块	
Fixed-size Chunking	固定大小分块	
Precision	精确率	
Baseline	基线	
Query Rewriting	查询改写	

英文原词	中文译名	缩写／符号
Query Expansion	查询扩展	
Representation	表示	
Vocabulary	词表	
Hypothetical Document Embeddings	假设文档嵌入	HyDE
Reciprocal Rank Fusion	倒数排名融合	RRF
Multi-hop Question	多跳问题	
Gating	门控	
Combine	合并	
Poisoning	投毒	
Training	训练	
Vector Database	向量数据库	
Large Language Model	大语言模型	LLM
Prompt Injection	提示词注入	
Access Control	访问控制	
Routing	路由	

目录导航

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

检索增强生成与上下文工程 · 第 6 页

关于本手册 · 第 8 页

例如，将表格压平成文本流，可能会让数值与对应的行、列标签分离；后续检索（Retrieval）可能返回表面正确、含义却错误的数字。UDA 直接以原始 PDF／HTML 为对象，特别强调了解析（Parsing）对端到端 RAG 形成的瓶颈。生产系统应在具有代表性的文档类型上评估提取质量，保留有用的源坐标与结构元数据；对表格或布局密集的材料，采用适合该来源的解析器或多模态方法。更好的嵌入向量（Embeddings）无法恢复解析阶段已经破坏的信息。[3] · 第 10 页

代表越来越广泛的信息。在查询（Query）时，它可以跨级别检索，旨在帮助需要整体或多步骤理解而不是局部跨度的问题。权衡是额外的索引成本和可能的摘要错误：抽象父节点本身是模型（Model）生成的，可以省略或扭曲细节。因此，常见的生产模式是使用更高级别的节点来导航或扩大召回率（Recall），但保留原始源段落的链接以供证据（Evidence）和引用（Citation）。当文档结构或问题范围需要时，层次结构很有价值；对于以局部事实查找为主的小型语料库来说，这是不必要的复杂性。 [7] · 第 12 页

16. 多查询检索（Multi-query Retrieval）和排名融合（Rank Fusion）何时可以提供帮助？为什么生成更多查询（Query）变体不会自动更好？ [高级] 单个短语可能会错过相关材料，因此系统可能会生成多个查询变体，对每个变体进行检索，采用并集并融合排名。倒数排名融合（Reciprocal Rank Fusion）在这里很有用，因为它结合了排名位置，而不需要校准来自不同结果列表的原始分数。当变体捕获真正不同的词汇或方面时，多查询检索可以扩大召回范围。它还会增加延迟（Latency）、重复候选者，并在多个搜索中放大错误的重写。因此，更多的变体并不能单调地提高质量。保留原始查询，删除重复结果，限制扩展预算，并测量每个添加查询的边际召回率（Recall）。如果语料库以精确标识符为主，或者原始查询已经是特定的，则额外的变体可能会增加大部分噪音。基本原则 - 查询扩展（Query Expansion）加上排名融合 - 比任何特定的“RAG-Fusion”配方更持久。 [9][11] · 第 14 页

不一致。对知识冲突的研究发现，在不同条件下，过度依赖外部证据（Evidence）和错误内部知识的持续存在。工程含义是显式编码源策略并对其进行测试。如果应用程序要求提供的语料库具有权威性，请明确说明，包括出处，并评估模型（Model）记忆的答案不同的反事实情况。如果语料库本身可能是错误的，那么盲目的上下文服从也是不安全的。对于高风险的冲突，返回分歧或寻求另一个来源，而不是强迫模型选择一个。 RAG 更改模型可用的证据；它不会自动校准模型如何仲裁证据。 [18][19] · 第 16 页

任何内容过滤器都不能证明系统安全。因此，高影响操作应设置确定性的授权与验证关卡。模型（Model）不应仅因检索文档的要求而获得新的权限。间接提示词注入研究，专门研究通过外部内容传入、而非由用户直接发出的恶意指令。[28][29][32] · 第 18 页

深度检查 · 第 20 页

对应原稿第 1 页

检索增强生成与上下文工程（Context Engineering）

核心概念、设计决策与失效模式

@techNmak

本手册中的问题

1. 什么是检索增强生成（Retrieval-augmented Generation）（RAG），它为语言模型（Model）添加了哪些模型参数本身无法提供的内容？ 2. 上下文工程（Context Engineering）比提示词工程（Prompt Engineering）更广泛，RAG 在其中的位置如何？ 3. 生产 RAG 管道的主要阶段是什么？为什么要单独评估解析（Parsing）、检索（Retrieval）、上下文组装（Context Assembly）和生成（Generation）？ 4. 什么时候应该使用 RAG 而不是长上下文提示、微调（Fine-tuning）或继续预训练（Continued Pretraining）？ 5. 为什么在检索开始之前文档解析就会失败，特别是对于 PDF、表格和结构化文档？ 6. 为什么 RAG 没有普遍正确的块大小（Chunk Size）？当块变小或变大时会发生什么变化？ 7. 块重叠（Chunk Overlap）实际上解决了什么问题，过多的重叠会如何损害检索和生成？ 8. 在固定大小、结构感知和语义分块（Semantic Chunking）之间应该如何选择？ 9. 上下文检索（Contextual Retrieval）和延迟分块（Late Chunking）如何尝试保留文档级上下文，它们有何不同？ 10. 什么是层次检索（Hierarchical Retrieval）？为什么类似 RAPTOR 的索引可以帮助处理长文档或需要多个抽象级别的问题？ 11. 你应该如何选择top-k，为什么检索更多的段落会使最终的答案变得更糟？ 12. 为什么上下文窗口（Context Window）内检索到的证据（Evidence）的顺序和位置会改变模型性能（Performance）？ 13. 检索后重排序（Reranking）有什么作用？为什么即使第一阶段检索器（Retriever）很强，它仍然有用？ 14. 为什么在检索之前重写或扩展用户的查询（Query）？查询转换如何会意外改变用户的意图？ 15. 什么是 HyDE，为什么模型生成的假设文档不能被视为证据？

16. 多查询检索（Multi-query Retrieval）和排名融合（Rank Fusion）何时可以提供帮助？为什么生成（Generation）更多查询（Query）变体不会自动更好？ 17. 为什么一次性检索（Retrieval）在多跳问题上会失败？交错检索与推理会发生什么变化？ 18. 每个请求都应该检索固定数量的段落吗？主动、适应性和纠正性 RAG 方法如何改变这一假设？ 19. 什么是上下文压缩（Context Compression）或选择性增强，压缩检索到的证据（Evidence）何时会降低系统的可靠性？ 20. RAG 系统应如何处理冗余、不相关和相互冲突的检索证据？ 21.当检索到的证据与模型（Model）的参数化知识（Parametric Knowledge）相冲突时会发生什么？模型将遵循哪个来源？ 22. 为什么在答案中附上引文并不能证明引用（Citation）的来源支持该主张或该模型实际上依赖于它？ 23.为什么即使检索器（Retriever）返回了正确的证据，RAG系统也会产生幻觉？ 24. 生产 RAG 系统应如何处理新鲜度、出处、更新、删除和重新索引（Index）？ 25. 应如何在 RAG 中强制执行权限，以便未经授权的文档永远不会成为模型上下文？ 26. 中毒或带有指令的文档如何攻击 RAG 系统？为什么检索到的文本应被视为不可信数据？ 27. 您应该使用哪些指标来评估检索、上下

文质量、生成、归因和端到端 RAG 系统？ 28. RAGAS、ARES 和 RAGChecker 实际评估什么？为什么自动 RAG 判断不是基准真值（Ground Truth）？ 29. GraphRAG 旨在解决什么问题，什么时候基于图的索引是不必要的或者比普通 RAG 更糟糕？ 30. 为不断变化的企业语料库设计一个生产 RAG 系统，具有权限、引用、多跳问题、长文档和严格的延迟（Latency） SLO。你如何证明每一层都有效？

关于本手册

对应原稿第 2 页

本手册重点介绍检索增强生成（Retrieval-augmented Generation）和提供给语言模型（Models）的上下文工程。它假设嵌入向量（Embeddings）、词法／稠密检索（Retrieval）、ANN 索引和重排序（Reranking）的机制是单独介绍的。这里的目标是检索之上的层：摄取、分块（Chunking）、查询转换、证据（Evidence）选择、上下文打包（Packing）、归因、安全性、评估和生产架构。个别论文的结果被表述为这些研究的结果，而不是普遍规律。

问题与解答

1. 什么是检索增强生成（Retrieval-augmented Generation）（RAG）？它为语言模型（Model）提供了哪些仅靠模型参数无法提供的能力？【入门】
RAG 在推理（Inference）时从外部知识源检索信息，用以增强生成（Generation）。模型仍提供其参数化知识（Parametric Knowledge）和语言生成能力，但提示词或模型计算会以检索到的非参数证据（Evidence）为条件。在最初的 RAG 建模中，检索器（Retriever）为文档赋予概率，生成器（Generator）则同时以输入和检索文档为条件。核心思想是：有用的知识不必只存储在模型权重中。
这样更容易更新外部知识，也为追溯来源提供了途径。但它不会自动保证答案符合事实：检索器可能漏掉证据，语料库本身可能有误，生成器也可能忽略或误用正确的上下文。因此，应将 RAG 理解为获取证据的架构，而不是消除幻觉（Hallucination）的开关。[1]

2. 上下文工程（Context Engineering）比提示词工程（Prompt Engineering）更广泛，RAG 在其中的位置如何？ [中级] 提示词工程主要是编写和组织指令。上下文工程更广泛：它是在推理（Inference）时呈现给模型（Model）的完整词元（Token）状态的设计 - 指令、检索（Retrieval）到的证据（Evidence）、工具结果、对话状态、元数据、示例和其他可能影响行为的信息。 RAG 是一种从外部语料库获取部分上下文的机制。如果检索到的段落顺序错误、冗余、过长、过时、未经授权或混有不明确的指令，那么一个好的 RAG 检索器（Retriever）仍然会产生一个糟糕的系统。相反，上下文工程可以在根本没有向量（Vector）检索的情况下存在；系统可以按需加载文件、使用结构化查询（Queries）、缓存上下文或工具输出。实际目标不是最大化上下文长度。它是提供一个足够小的、高信号的上下文，足以完成任务并且模型可以安全使用。 [2]

3. 生产 RAG 管道的主要阶段是什么？为什么要单独评估解析（Parsing）、检索（Retrieval）、上下文组装（Context Assembly）和生成（Generation）？ [中级-系统设计] 生产管道通常至少有以下阶段：源摄取和解析；文档分割；索引；查询（Query）解释；候选检索；可选的融合或重排序（Reranking）；授权过滤；上下文选择和排序；生成；归因;和记录/评估。每个阶段都会造成不同的失败。即使检索完美，表也可能被错误地解析。候选集中可能不存在正确的段落。重排序器（Reranker）可能会丢弃它。该段落可能出现在提示中，但生成器（Generator）可能会忽略它。答案可能是正确的，但引用了错误的来源。仅端到端的准确性（Accuracy）就隐藏了这些区别。例如，UDA 基准明确将解析、检索、生成和端到端文档分析作为单独的设计选择进行研究。工程启示是保留足够的中间产物 - 解析的单元、检索的 ID 和分数、重

排序的上下文、最终提示、引用 - 来逐层测试管道。 [3]

4. 什么时候应该使用 RAG 而不是长上下文提示、微调（Fine-tuning）或继续预训练（Continued Pretraining）？ [高级 - 判断] 当任务依赖的语料库太大、太动态、权限太敏感或来源太关键而无法完全放入提示或模型（Model）权重时，请使用 RAG。当相关语料库足够小以经济地适合并且模型可以可靠地使用它时，长上下文提示可以更简单。微调更适合改变行为、风格、任务能力或学习模式；对于必须更新或引用的经常变化的事实记录来说，它是一个糟糕的替代品。持续的预训练可以使模型适应某个领域，但它也会改变权重并且不会给出文档级的出处。不存在通用的 RAG 与长上下文的胜利者。 2024 年的一项研究发现，在一些资源充足的设置下，长上下文平均更强，而 2025 年 LaRA 基准测试（Benchmark）发现最佳选择取决于模型、上下文长度、任务类型和检索（Retrieval）特征。基于测量的质量、成本、延迟（Latency）、新鲜度和治理要求的路由。 [26][27]

5. 为什么在检索（Retrieval）开始之前文档解析（Parsing）就会失败，特别是对于 PDF、表格和结构化文档？ [中级 - 生产] 检索只能搜索保留的摄取内容。真实的 PDF 和 Web 文档包含标题、栏目、脚注、表格、页面结构、阅读顺序歧义、扫描文本，有时还包含其结构在简单文本提取下消失的图形。因此，解析器可以在分块（Chunking）或嵌入（Embedding）之前破坏证据（Evidence）

例如，将表格压平成文本流，可能会让数值与对应的行、列标签分离；后续检索（**Retrieval**）可能返回表面正确、含义却错误的数字。**UDA** 直接以原始 **PDF／HTML** 为对象，特别强调了解析（**Parsing**）对端到端 **RAG** 形成的瓶颈。生产系统应在具有代表性的文档类型上评估提取质量，保留有用的源坐标与结构元数据；对表格或布局密集的材料，采用适合该来源的解析器或多模态方法。更好的嵌入向量（**Embeddings**）无法恢复解析阶段已经破坏的信息。**[3]**

对应原稿第 3 页

6. 为什么 RAG 没有普遍正确的块大小（Chunk Size）？当块变小或变大时会发生什么变化？[中级 - 权衡] 块大小控制着真正的权衡。非常小的块可以精确地定位事实并减少不相关的词元（Tokens），但它们可以将定义、限定词、标题或多句推理与它们解释的证据（Evidence）分开。大块保留了更多的本地上下文，但它们的嵌入向量（Embeddings）可能会模糊多个主题，检索（Retrieval）的选择性可能会降低，并且生成器（Generator）会收到更多不相关的词元。因此，最佳大小取决于文档结构、嵌入向量模型（Model）、查询（Queries）粒度、检索策略和预期答案的类型。最近的文档分割工作同样发现分割对检索和问答有重大影响，但没有建立一个通用的词元计数。可靠的工作流程是生成或标注代表性查询、扫描多个块大小/策略、测量检索证据覆盖范围和端到端答案质量，以及检查边界故障。复制的默认值（例如“512 个词元”）是起始配置，而不是工程结论。

[4]

7. 块重叠（Chunk Overlap）实际上解决了什么问题，过多的重叠会如何损害检索（Retrieval）和生成（Generation）？[中级]重叠主要是一种边界对冲：如果一个重要的跨度跨越了一个块的边界，重复一些相邻的文本可以让至少一个块包含足够的证据（Evidence）。成本就是重复。更多的重叠会创建更多的向量或索引（Index）条目，增加摄取和存储，并且可以使几个几乎相同的块占据 top-k 结果。这会浪费上下文预算，并且可能会在生成过程中过度强调重复的证据。它还可以使评估在块级别上看起来更好，因为同一事实多次出现而无需增加独立证据。不存在始终最佳的通用百分比。当分割方法产生有害的边界切割时使用重叠，然后衡量它是否真正提高了证据召回率（Recall）。结构感知分割、父子检索、上下文化块或在初始命中后检索相邻部分有时可以保持连续性并减少重复。目标是提供足够的证据，而不是最大限度地重复使用词元（Token）。

8. 在固定大小、结构感知和语义分块（Semantic Chunking）之间应该如何选择？[高级 - 判断] 固定大小的分块成本低、具有确定性并且是有用的基线（Baseline）。结构感知分块（Structure-aware Chunking）遵循

节、段落、代码块或表单元等边界，并且当源格式包含有意义的层次结构时通常更可取。语义分块尝试在主题变化的地方设置边界，但增加了模型（Model）或嵌入（Embedding）依赖性，并且不能保证在每个语料库上都优于更简单的方法。 ACL 2025 关于文档分割的研究结果表明，过大和过小的片段会产生不同的故障模式，并且分割可以改善检索（Retrieval）和端到端 QA，而 2026 年对学术文本的评估还发现，经 过测试的基于集群的语义分块方法并没有优于更简单的固定大小或递归策略。按来源结构和测量证据（ Evidence）检索进行选择。保留稳定的文档 ID 和父层次结构，以便系统可以检索狭窄的子文档以实现精 确性，同时在需要更广泛的上下文时仍扩展到其部分或文档。 [4][31]

9. 上下文检索（Contextual Retrieval）和延迟分块（Late Chunking）如何尝试保留文档级上下文，它们有何不同？ [高级] 这两种技术都解决了同样的问题：对于看到整个文档的人来说，一个块可能是本地可以理解的，但单独嵌入（Embedding）时却是模糊的。 Anthropic 的上下文检索从完整文档中生成一个简短的特定于块的描述，并在构建语义和词汇索引之前添加该上下文。延迟分块采用不同的路线：长上下文嵌入模型（Model）首先计算词元（Tokens）表示，而词元可以处理更大的文档，然后才将词元表示汇集到块向量中。前者改变了被索引的文本，可以与普通的下游嵌入/BM25管道一起工作；后者改变了嵌入计算中分块（Chunking）发生的位置，并且需要支持该方法的模型。两者都不是正常分块的通用替代品。两者都会增加成本或模型假设，并且应该在实际语料库上进行评估。 Anthropic 报告的收益是其自己实验的结果，而不是一般保证。 [5][6]

10. 什么是层次检索（Hierarchical Retrieval）？ 为什么类似 RAPTOR 的索引可以帮助处理长文档或需要多个抽象级别的问题？ [高级] 平面块检索（Retrieval）仅以一种粒度表示长文档。层次检索添加了多个级别 - 例如段落、部分、摘要或集群 - 因此查询（Query）可以检索特定证据（Evidence）或更广泛的抽象。 RAPTOR 递归地嵌入、聚类和汇总文本以构建树，其较高节点

代表越来越广泛的信息。在查询（**Query**）时，它可以跨级别检索，旨在帮助需要整体或多步骤理解而不是局部跨度的问题。权衡是额外的索引成本和可能的摘要错误：抽象父节点本身是模型（**Model**）生成的，可以省略或扭曲细节。因此，常见的生产模式是使用更高级别的节点来导航或扩大召回率（**Recall**），但保留原始源段落的链接以供证据（**Evidence**）和引用（**Citation**）。当文档结构或问题范围需要时，层次结构很有价值；对于以局部事实查找为主的小型语料库来说，这是不必要的复杂性。 **[7]**

对应原稿第 4 页

11. 你应该如何选择top-k，为什么检索（Retrieval）更多的段落会使最终的答案变得更糟？ [中级 - 上下文] Top-k 是上下文预算决策，而不是检索常量。提高 k 可能会提高支持证据（Evidence）出现的机会，特别是当检索器（Retriever）的排名不完美时。但每一个额外的段落都可能添加不相关、冗余、冲突或质量较低的文本。这会增加投入成本，并可能使生成器（Generator）更难识别证据。 Self-RAG 特别激励自适应检索，部分原因是盲目检索固定数量的段落可能没有帮助。正确的 k 取决于候选召回曲线、重排序（Reranking）质量、平均段落大小、模型（Model）上下文行为和答案类型。评估多个深度的检索召回率（Recall），然后评估使用实际打包上下文的生成（Generation）。通常，正确的设计是以低成本检索更多候选者，对它们重排序，删除重复项，并将较小的证据集传递给模型，而不是简单地增加提示

k. [14]

12. 为什么上下文窗口（Context Window）内检索（Retrieval）到的证据（Evidence）的顺序和位置会改变模型（Models）性能（Performance）？ [中级 - 上下文] 长上下文模型不一定能同样好地使用所有位置。Lost in the Middle 在多文档 QA 和键值检索方面表现出显着的位置敏感性：该研究中的模型通常在相关信息出现在开头或结尾附近时最强，而在相关信息出现在中间时则较弱。确切的模式取决于模型和任务，因此不应将其变成通用规则，例如“始终将最好的块放在最后”。持久的教训是上下文打包（Packing）是模型接口的一部分。故意对最重要的证据进行排名，对相关段落进行分组，避免不必要的填充，并测试代表性任务的排列。重排序还可能与引文、时间推理和矛盾来源相互作用。宣称支持大上下文窗口的模型告诉您它可以接受多少个词元（Tokens），而不是每个词元位置都会对答案质量做出同等贡献。 [8]

13. 检索（Retrieval）后重排序（Reranking）有什么作用？为什么即使第一阶段检索器（Retriever）很强，它仍然有用？ [中级 - 系统设计] 第一阶段检索经过优化，可以有效地搜索大型语料库，因此它通常使用更便宜的词汇或嵌入（Embedding）分数。重排序使用更强的相关性模型（Models）或更丰富的查询（Query）文档交互在更小的候选集上花费更多的计算。在 RAG 中，它的价值不仅仅是更高的搜索度量：

它确定哪些证据（Evidence）消耗有限的生成器（Generator）上下文。更好的重排序器（Reranker）可以删除看似合理但错误的块，并将真正支持的段落移至提示中。代价是额外的延迟（Latency）和成本，并且重排序仅对第一阶段检索的文档有帮助。因此，一个有用的设计是测量候选深度的第一阶段证据召回率（Recall）、多个截止点的重排序增益以及端到端答案质量。例如，Anthropic 的上下文检索（Contextual Retrieval）实验报告了添加重排序后检索的额外改进，但这些数字特定于其评估的语料库和模型。 [5]

14. 为什么在检索（Retrieval）之前重写或扩展用户的查询（Queries）？查询转换如何会意外改变用户的意图？ [高级] 用户查询通常不明确、简短或使用与语料库不同的词汇来表达。查询扩展（Query Expansion）或重写可以添加同义词、消除歧义的术语、可能的实体或更类似于文档的表示（Representation）。例如，Query2doc 生成伪文档并使用它们来扩展查询，从而改进其报告的实验中的检索。风险在于语义漂移：语言模型（Model）可能会添加请求中未包含的假设、错误地缩小不明确的查询范围或引入幻觉实体。强大的系统通常会保留原始查询，记录每个转换，并融合或比较结果，而不是默默地替换用户的意图。查询重写应该根据召回增益和重写改变含义的失败情况来评估。它是一个检索工具，而不是重新解释用户任务的权限。 [9]

15. 什么是 HyDE，为什么模型（Model）生成的假设文档不能被视为证据（Evidence）？ [高级] HyDE - 假设文档嵌入（Hypothetical Document Embeddings） - 要求语言模型生成一个假设文档来回答查询（Query），嵌入（Embedding）生成的文本，并使用嵌入来检索（Retrieval）附近的真实文档。生成的文档可能包含虚假详细信息； HyDE 论文明确承认它是“假的”。它的作用是将查询转移到类似于相关文档的表示（Representation）中，而不是提供事实证据。答案生成器（Generator）仍应在检索到的语料库文档中论证声明。这种区别在操作上很重要：如果系统意外地将假设文本包含为可信上下文或引用它，检索辅助工具就会成为幻觉来源。当标记的相关性数据不可用且零样本稠密检索较弱时，HyDE 最有趣，但它的好处取决于工作负载，并且它在检索之前添加了 LLM 调用。 [10]

16. 多查询检索（Multi-query Retrieval）和排名融合（Rank Fusion）何时可以提供帮助？为什么生成更多查询（Query）变体不会自动更好？ [高级] 单个短语可能会错过相关材料，因此系统可能会生成多个查询变体，对每个变体进行检索，采用并集并融合排名。倒数排名融合（Reciprocal Rank Fusion）在这里很有用，因为它结合了排名位置，而不需要校准来自不同结果列表的原始分数。当变体捕获真正不同的词汇或方面时，多查询检索可以扩大召回范围。它还会增加延迟（Latency）、重复候选者，并在多个搜索中放大错误的重写。因此，更多的变体并不能单调地提高质量。保留原始查询，删除重复结果，限制扩展预算，并测量每个添加查询的边际召回率（Recall）。如果语料库以精确标识符为主，或者原始查询已经是特定的，则额外的变体可能会增加大部分噪音。基本原则 - 查询扩展（Query Expansion）加上排名融合 - 比任何特定的“RAG-Fusion”配方更持久。 **[9][11]**

对应原稿第 5 页

17. 为什么一次性检索（Retrieval）在多跳问题（Multi-hop Question）上会失败？交错检索与推理会发生什么变化？ [高级] 对于多跳问题，第二次搜索所需的信息可能不存在于原始问题中。因此，即使语料库中存在所有必需的事实，一次检索调用也可能会失败。 IRCoT 演示了一种将检索与推理交织在一起的方法：早期检索的证据（Evidence）和中间推理有助于制定下一步要检索的内容，然后新证据支持后面的推理步骤。架构启示是查询（Query）分解和有状态检索，而不是思维链字符串本身必须公开或信任。生产系统可以将任务分解为子问题，检索每个子问题，维护明确的证据状态，并验证最终答案是否得到源联合的支持。迭代检索需要更多调用，并且可能会级联早期错误的分解，因此仅当任务实际需要时才应调用多跳行为，并根据跳级证据覆盖率进行评估。 [12]

18. 每个请求都应该检索（Retrieval）固定数量的段落吗？主动、适应性和纠正性 RAG 方法如何改变这一假设？ [高级] 修复了“每个请求检索一次 top-k”仅为一个 RAG 策略的问题。当即将出现的内容显得不确定时，FLARE 在长格式生成（Generation）期间进行检索。 Self-RAG 训练模型（Model）发出反思信号，控制何时检索以及如何批评检索到的证据（Evidence）和生成。 CRAG 添加了一个检索评估器，并在检索到

的文档显得薄弱时选择纠正措施，包括知识精炼以及本文设置中的外部网络搜索。这些是不同的方法，但 它们有一个共同的原则：检索可以是有条件的和反馈驱动的。权衡是复杂性。门控（Gating）模型可能会 犯错误，迭代检索会增加延迟（Latency），而网络回退会改变信任和新鲜度模型。当检索所有内容的成 本很高或任务的知识需求差异很大时，请使用自适应检索；不要假设命名方法会原封不动地传输到每个系 统。 [13][14][15]

19. 什么是上下文压缩（Context Compression）或选择性增强，压缩检索到的证据（Evidence）何时会降低系统的可靠性？ [高级] 检索到的段落可能比实际需要的证据长得多。上下文压缩试图保留高价值内容，同时减少词元（Tokens）。 RECOMP 训练提取或抽象压缩器，并在检索到的文档未添加有用信息时输出空字符串，将压缩与选择性增强相结合。 LongLLMLingua 压缩长提示以降低成本并解决信息密度/位置影响。失败模式是明显但严重的：压缩可以删除答案所依赖的限定符、数字、否定、异常或出处提示。抽象压缩还会在主生成器（Generator）运行之前引入生成（Generation）错误。将压缩上下文视为派生表示（Representation）；保留原始段落以进行验证和引用，并在多个压缩比下评估答案质量。仅当节省的词元和延迟（Latency）超过预处理成本和丢失的证据时，压缩才是有益的。 [16][17]

20. RAG 系统应如何处理冗余、不相关和相互冲突的检索（Retrieval）证据（Evidence）？ [高级-可靠性] 尽可能在最终提示之前处理冗余、无关和冲突。对几乎相同的块进行重复数据删除，这样重复的副本就不会消耗上下文或产生错误的证实感。重排序或过滤不相关的候选者，而不是要求生成器（Generator）在嘈杂的提示中发现相关性。对于相互矛盾的证据，保留来源身份、时间戳和出处；不要默默地将矛盾合并到一个声明中。对冲突检索的研究表明，模型（Models）对不一致的证据反应不佳，并且在特定的实验设置中可能表现出频率效应或确认效应。可靠的应用程序应该在重大冲突影响答案时将其暴露出来，根据明确的政策优先选择权威或当前的来源，并在冲突无法解决时允许拒答。“检索到的块的大多数”不是事实标准，因为重复或有毒的内容可以创建人为的多数。 [18][19]

21.当检索到的证据（Evidence）与模型（Model）的参数化知识（Parametric Knowledge）相冲突时会发生什么？模型将遵循哪个来源？ [高级-可靠性] 没有通用的优先规则。根据模型、提示、主题熟悉程度、证据质量和冲突设置，模型可能会遵循检索到的上下文，坚持其参数先验，将两者混合，或者成为

不一致。对知识冲突的研究发现，在不同条件下，过度依赖外部证据（**Evidence**）和错误内部知识的持续存在。工程含义是显式编码源策略并对其进行测试。如果应用程序要求提供的语料库具有权威性，请明确说明，包括出处，并评估模型（**Model**）记忆的答案不同的反事实情况。如果语料库本身可能是错误的，那么盲目的上下文服从也是不安全的。对于高风险的冲突，返回分歧或寻求另一个来源，而不是强迫模型选择一个。 **RAG** 更改模型可用的证据；它不会自动校准模型如何仲裁证据。 **[18][19]**

对应原稿第 6 页

22. 为什么在答案中附上引文并不能证明引用（Citation）的来源支持该主张或该模型（Model）实际上依赖于它？ [高级 - 归因] 引文可能会以多种不同的方式失败。所引用的段落可能不包含主张；一项主张可能根本没有引用；引文可能仅支持复合句的一部分；或者模型在从参数记忆中生成（Generation）答案后可能会生成看起来正确的引文。 ALCE 评估被引生成，并将答案质量与引文质量分开。后来的工作将引用正确性（来源是否支持该主张）与引用忠实性（模型是否真正依赖于引用的证据（Evidence）而不是事后合理化）区分开来。在生产中，保留从声明到源跨度的映射，检查引文蕴涵或支持，测量外部可验证声明的覆盖范围，并使源开放对用户来说变得容易。句子旁边的 URL 是元数据，而不是证据。归因是其自身的评价目标。 [20][21]

23.为什么即使检索器（Retriever）返回了正确的证据（Evidence），RAG系统也会产生幻觉？ [中级 - 可靠性] 正确检索（Retrieval）仅解决证据访问问题。生成器（Generator）仍然可能忽略该段落，误读表格，犯算术或推理错误，错误地组合证据，遵循不相关的相邻段落，或者更喜欢其参数先验。它还可以围绕其他受支持的核心答案生成不受支持的详细信息。这就是为什么 RAG 评估框架将上下文相关性与答案忠实性/相关性/正确性分开，而不是将检索召回视为最终指标。一个有用的调试测试是 Oracle 上下文实验：仅向生成器提供已知的支持证据。如果答案仍然失败，检索就不再是主要瓶颈。相反，如果 oracle 上下文工作但正常的 RAG 失败，请检查解析（Parsing）、检索、重排序（Reranking）或打包（Packing）。 RAG 可以减少一些事实错误，但它并不能消除将生成器评估为推理和证据使用组件的需要。 [22][23][24]

24. 生产 RAG 系统应如何处理新鲜度、出处、更新、删除和重新索引（Index）？ [高级 - 生产] RAG 仅与源到索引管道一样新鲜。存储稳定的源 ID、版本或修订标识符、时间戳、出处和摄取状态。更新应该传播到所有派生的产物：解析的文本、块、嵌入向量（Embeddings）、词汇索引、图形结构、缓存和引文映射。删除尤其重要，因为删除源记录但保留旧的嵌入向量或缓存的提示可能会继续暴露陈旧或禁止的文本。根据新鲜度 SLO 确定更新是事件驱动的、批量的还是定期的，并明确测量摄取延迟。在迁移期间，支持双索引或版本化部署，以便查询（Queries）不会混合不兼容的表示。非参数记忆最初的吸引力在于可以

更新知识而无需重新训练生成器（Generator）；只有当工程系统真正保持索引的新鲜度和沿袭性时，这种优势才存在。 [1]

25. 如何在 RAG 中强制执行权限，以使未经授权的文档永远不会成为模型（Model）上下文？ [高级 - 安全性] 授权必须是数据访问边界，而不是模型的自然语言指令。在将未经授权的内容组装到模型上下文中之前，系统应确定用户的有效权限并限制候选文档。根据架构的不同，这可能意味着搜索层中的权限感知过滤、以调用者身份查询源系统，或者在提示构建之前进行检索（Retrieval）后授权 - 但绝不依赖 LLM 来“忽略”不允许其查看的文档。权限还必须传播到缓存的结果、摘要、图形节点和派生索引。最近的企业访问控制研究主张在检索和生成（Generation）边界上执行确定性执行，因为提示词清理和输出过滤是概率性保障措施，而不是授权的替代品。记录哪些受保护对象影响每个答案，以便访问决策可审核。 [30]

26. 中毒或带有指令的文档如何攻击 RAG 系统？为什么检索（Retrieval）到的文本应被视为不可信数据？ [高级 - 安全性] RAG 语料库扩大了攻击面，因为检索到的文本是外部输入。 PoisonedRAG 证明，将恶意文档注入知识库可以引导有针对性的查询（Queries）的答案，并且后来的安全基准测试显示了具有误导性或对抗性上下文的更广泛的稳健性问题。即使没有优化的中毒攻击，普通文档也可能包含与系统策略冲突的类似指令的文本。因此，检索到的内容应被视为不受信任的数据：保留来源和信任级别，在提示设计中将指令与证据（Evidence）分开，限制哪些来源可以进入敏感工作流程，在适当的情况下扫描或验证摄取，在模型（Model）之外强制执行工具权限，并监控可疑的检索模式。不

任何内容过滤器都不能证明系统安全。因此，高影响操作应设置确定性的授权与验证关卡。模型（**Model**）不应仅因检索文档的要求而获得新的权限。间接提示词注入研究，专门研究通过外部内容传入、而非由用户直接发出的恶意指令。**[28][29][32]**

对应原稿第 7 页

27. 您应该使用哪些指标来评估检索（Retrieval）、上下文质量、生成（Generation）、归因和端到端 RAG 系统？ [高级-评估] 多层评估。对于检索，请在合适的情况下使用面向证据（Evidence）的措施，例如 Recall@k、Precision@k、MRR/nDCG，以及实际查询（Queries）类型的延迟（Latency）和覆盖率。对于组装的上下文，衡量是否存在支持证据、剩余多少不相关或重复的材料，以及所需的权限/出处是否正确。对于生成，区分答案的正确性、对所提供证据的忠实度以及答案的相关性。对于归因答案，衡量引文支持度和引文完整性。端到端评估还应包括拒答行为、新鲜度、安全案例、延迟、成本和特定于任务的业务成果。单个总分无法告诉您要修复哪个模块。为困难类别（例如多跳、精确标识符、冲突源、陈旧文档、表和权限过滤查询）构建切片级结果。 RAGAS、ARES 和 RAGChecker 都反映了这种多维视图。 [22][23][24]

28. RAGAS、ARES 和 RAGChecker 实际评估什么？为什么自动 RAG 判断不是基准真值（Ground Truth）？ [高级 - 评估] RAGAS 使用自动评估提出了上下文质量、答案忠实度和答案相关性等维度的无参考指标。 ARES 生成合成训练（Training）数据，训练轻量级评估器的上下文相关性、答案可信度和答案相关性，并使用小型人工注释验证集和预测辅助推断来改进总体估计和置信区间。 RAGChecker 使用声明级操作以及更大的检索器（Retriever）和生成器（Generator）诊断指标套件，然后元评估与人类判断的相关性。它们是有用的工具，而不是可互换的真理机器。基于 LLM 的评估器继承了模型（Model）偏差、提示敏感性、领域不匹配和错误。合成问题可能会歪曲生产流量。使用自动评估迭代速度，但根据真实领域中人工标记的示例对其进行校准，并定期检查分歧。仅当其测量行为已针对您的用例进行验证时，指标才值得信赖。 [22][23][24]

29. GraphRAG 旨在解决什么问题，什么时候基于图的索引（Index）是不必要的或者比普通 RAG 更糟糕？ [高级] GraphRAG 针对的问题的答案取决于语料库中的关系或全局模式，而不是一两个局部相似的段落。 Microsoft 的 GraphRAG 构建实体/关系图，将其聚类为社区，并创建社区摘要；然后，全局搜索可以对这些摘要进行推理，而本地模式则使用以实体为中心的上下文。已发表的作品特别关注语料库级别的意义建构问题，而传统的本地 RAG 不太适合这些问题。 GraphRAG 不是通用升级。索引需要实体/关系提取和 LLM 生成的摘要，这会增加成本、延迟（Latency）、存储和新的错误模式。对于直接事实查找、精确标识符或小型文档集合，正常的词法／稠密检索（Retrieval）可能更简单、更好。当查询（Query）确实需要关系、主题或多文档聚合时，请使用图形结构，并保留返回原始证据（Evidence）的链接，因为图形节点和摘要是派生的产物。 [25]

30. 为持续变化的企业语料库设计生产级 RAG 系统，需要支持权限、引用、多跳问题、长文档，并满足严格的延迟（Latency） SLO。如何证明每一层都有效？【高级・系统设计】

从治理要求与工作负载出发。数据摄取应保留来源 ID、版本、结构和访问控制列表（ACL）；解析（Parsing）必须在具有代表性的 PDF 与表格上测试。先建立简单的词法＋稠密检索（Retrieval）基线（Baseline），测量证据（Evidence）召回率（Recall）；只有确实改善候选结果到最终上下文的选择时，才增加重排序（Reranking）。根据文档结构与评估结果选择分块（Chunking）方式；只有在确实解决相应失败案例时，才使用层次检索或上下文化检索。

在组装提示词之前执行授权检查。将多跳问题交给迭代检索；语料库层面的主题问题可考虑图方法或层次方法；对于规模较小、边界明确的语料库，直接测试长上下文。上下文打包（Packing）应去重、保留来源、有意识地排列高价值证据，并满足明确的词元（Token）预算。生成器（Generator）应引用源文档中的具体片段，遇到证据不足或互相冲突时拒答。

安全测试必须覆盖投毒文档、带有指令的文档及权限边界。使用各层专门的评估、端到端细分结果、延迟／成本 SLO、内容更新滞后和事件日志来验证系统。每个新增组件都必须有实测的、由它解决的失败模式；否则，它只是架构上的装饰。[3][25][26][27][30]

深度检查

对应原稿第 8 页

区域 强烈的理解是什么样的

分块（Chunking）解释了证据（Evidence）粒度权衡，并提出了特定于语料库的评估，而不是命名神奇的词元（Token）大小。上下文打包（Packing） 将 top-k、重复数据删除、排序和压缩视为词元预算优化问题，而不是独立切换。可靠性 将检索（Retrieval）失败与生成（Generation）失败分开，并了解正确的上下文仍然可以产生不受支持的答案。归因 区分答案的正确性、支持度、引文覆盖率以及对证据的真正依赖。安全性 在上下文组装（Context Assembly）之前强制执行授权，并将检索到的文档视为不受信任的输入，而不是指令。评估使用特定于层的指标和代表性切片，然后根据人工标记的生产示例验证自动判断。

其他值得了解的概念

提示缓存和上下文重用。有时可以缓存重复的大型上下文以降低成本或延迟（Latency），但缓存身份和授权成为正确性的一部分。即时代理搜索。代理可以保留轻量级引用并仅在需要时加载信息，而不是预加载所有潜在相关的上下文；这属于更广泛的上下文工程问题。结构化和多模式 RAG。 SQL、知识图、表格、图像和代码通常需要保留结构的检索（Retrieval）或工具接口，而不是将所有内容扁平化为文本块。拒答和可回答性。生产系统应区分“未找到相关证据（Evidence）”和“找到证据但不充分/冲突”，并测试模型（Model）是否适当拒答。

参考文献

[1] Patrick Lewis et al. 面向知识密集型自然语言处理任务的检索增强生成（Retrieval-augmented Generation）。NeurIPS 2020。

[2] Anthropic. 面向 AI 智能体的有效上下文工程（Context Engineering）。工程博客，2025。

[3] Yulong Hui, Yao Lu, Huanchen Zhang. UDA：面向真实世界文档分析中检索增强生成的基准测试（Benchmark）套件。NeurIPS 2024 数据集与基准赛道。

[4] Zhitong Wang et al. 文档分割对检索增强生成至关重要。Findings of ACL 2025。

[5] Anthropic. 上下文检索（Contextual Retrieval）介绍。工程博客，2024。

[6] Michael Gunther, Isabelle Mohr, Daniel James Williams, Bo Wang, Han Xiao. 延迟分块（Late Chunking）：利用长上下文嵌入向量（Embeddings）模型（Models）生成（Generation）带上下文的块嵌入向量。arXiv:2409.04701，2024。

[7] Parth Sarthi et al. RAPTOR：用于树状组织检索（Retrieval）的递归抽象处理。ICLR 2024。

[8] Nelson F. Liu et al. 迷失在中间：语言模型如何利用长上下文。TACL 2023。

[9] Liang Wang, Nan Yang, Furu Wei. Query2doc：利用大语言模型扩展查询（Query）。EMNLP 2023。

[10] Luyu Gao, Xueguang Ma, Jimmy Lin, Jamie Callan. 无相关性标签的精确零样本稠密检索（HyDE）。ACL 2023。

[11] Gordon V. Cormack, Charles L. A. Clarke, Stefan Buettcher. 倒数排名融合（Reciprocal Rank Fusion）优于 Condorcet 方法及单独的排名学习方法。SIGIR 2009。
[12] Harsh Trivedi et al. 面向知识密集型多步骤问题，将检索与思维链推理交错进行。ACL 2023。
[13] Zhengbao Jiang et al. 主动检索增强生成（FLARE）。EMNLP 2023。

原题名与书目信息：[1] Patrick Lewis et al. Retrieval-Augmented Generation for Knowledge- Intensive NLP Tasks. NeurIPS 2020. [2] Anthropic. Effective context engineering for AI agents. Engineering blog, 2025. [3] Yulong Hui, Yao Lu, Huanchen Zhang. UDA: A Benchmark Suite for Retrieval Augmented Generation in Real-world Document Analysis. NeurIPS 2024 Datasets and Benchmarks Track. [4] Zhitong Wang et al. Document Segmentation Matters for Retrieval- Augmented Generation. Findings of ACL 2025. [5] Anthropic. Introducing Contextual Retrieval. Engineering blog, 2024. [6] Michael Gunther, Isabelle Mohr, Daniel James Williams, Bo Wang, Han Xiao. Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models. arXiv:2409.04701, 2024. [7] Parth Sarthi et al. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR 2024. [8] Nelson F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2023. [9] Liang Wang, Nan Yang, Furu Wei. Query2doc: Query Expansion with Large Language Models. EMNLP 2023. [10] Luyu Gao, Xueguang Ma, Jimmy Lin, Jamie Callan. Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). ACL 2023. [11] Gordon V. Cormack, Charles L. A. Clarke, Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009. [12] Harsh Trivedi et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. ACL 2023. [13] Zhengbao Jiang et al. Active Retrieval Augmented Generation (FLARE). EMNLP 2023.

来源带来复杂的知识冲突：重新校准模型（Models），使其反映相互冲突的证据（Evidence）。EMNLP 2022。
[19] Zhuoran Jin et al. 知识之间的拉锯战：探索并解决检索（Retrieval）增强语言模型中的知识冲突。LREC-COLING 2024。
[20] Tianyu Gao, Howard Yen, Jiatong Yu, Danqi Chen. 使大语言模型（Large Language Model）生成（Generation）带有引用的文本（ALCE）。EMNLP 2023。
[21] Jonas Wallat, Maria Heuss, Maarten de Rijke, Avishek Anand. RAG 归因中的正确性不等于忠实性。ICTIR 2025。
[22] Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert. RAGAS：检索增强生成（Retrieval-augmented Generation）的自动评估。EACL 2024 系统演示。
[23] Jon Saad-Falcon, Omar Khattab, Christopher Potts, Matei Zaharia. ARES：检索增强生成系统的自动评估框架。NAACL 2024。
[24] Dongyu Ru et al. RAGChecker：诊断检索增强生成的细粒度框架。NeurIPS 2024 数据集与基准赛道。
[25] Darren Edge et al. 从局部到全局：利用图 RAG 进行面向查询（Query）的摘要。arXiv:2404.16130，2025 年修订。
[26] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, Michael Bendersky. 检索增强生成还是长上下文大语言模型？一项综合研究与混合方法。EMNLP 2024 行业赛道。
[27] Kuan Li et al. LaRA：检索增强生成与长上下文大语言模型的基准测试——长上下文或 RAG 路由（Routing）均无万能解法。ICML 2025。
[28] Wei Zou, Runpeng Geng, Binghui Wang, Jinyuan Jia. PoisonedRAG：针对大语言模型检索增强生成的知识破坏攻击。arXiv:2402.07867，2024。
[29] Xun Liang et al. SafeRAG：大语言模型检索增强生成的安全性基准测试。ACL 2025。
[30] Shashank Shreedhar Bhatt et al. 企业 AI 必须执行感知参与者身份的访问控制（Access Control）。arXiv:2509.14608，2025。

[31] Valentin J. J. Kreileder, Johannes Reisinger, Andreas Fischer. 面向学术文本，评估检索增强生成的分块（Chunking）策略。arXiv:2607.01852，2026。

[32] Yulin Chen, Haoran Li, Yuan Sui, Yufei He, Yue Liu, Yangqiu Song, Bryan Hooi. 能否检测并移除间接提示词注入（Prompt Injection）攻击？ACL 2025。

原题名与书目信息：Sources Bring Complex Knowledge Conflicts: Recalibrating Models to Reflect Conflicting Evidence. EMNLP 2022. [19] Zhuoran Jin et al. Tug-of-War between Knowledge: Exploring and Resolving Knowledge Conflicts in Retrieval-Augmented Language Models. LREC-COLING 2024. [20] Tianyu Gao, Howard Yen, Jiatong Yu, Danqi Chen. Enabling Large Language Models to Generate Text with Citations (ALCE). EMNLP 2023. [21] Jonas Wallat, Maria Heuss, Maarten de Rijke, Avishek Anand. Correctness is not Faithfulness in RAG Attributions. ICTIR 2025. [22] Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. EACL 2024 System Demonstrations. [23] Jon Saad-Falcon, Omar Khattab, Christopher Potts, Matei Zaharia. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. NAACL 2024. [24] Dongyu Ru et al. RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation. NeurIPS 2024 Datasets and Benchmarks Track. [25] Darren Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, revised 2025. [26] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, Michael Bendersky. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. EMNLP 2024 Industry Track. [27] Kuan Li et al. LaRA: Benchmarking Retrieval-Augmented Generation and Long-Context LLMs - No Silver Bullet for LC or RAG Routing. ICML 2025. [28] Wei Zou, Runpeng Geng, Binghui Wang, Jinyuan Jia. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. arXiv:2402.07867, 2024. [29] Xun Liang et al. SafeRAG: Benchmarking Security in Retrieval- Augmented Generation of Large Language Model. ACL 2025. [30] Shashank Shreedhar Bhatt et al. Enterprise AI Must Enforce Participant-Aware Access Control. arXiv:2509.14608, 2025. [31] Valentin J. J. Kreileder, Johannes Reisinger, Andreas Fischer. Evaluating Chunking Strategies for Retrieval-Augmented Generation on Academic Texts. arXiv:2607.01852, 2026. [32] Yulin Chen, Haoran Li, Yuan Sui, Yufei He, Yue Liu, Yangqiu Song, Bryan Hooi. Can Indirect Prompt Injection Attacks Be Detected and Removed? ACL 2025.

[14] Akari Asai et al. Self-RAG：通过自我反思学习检索（Retrieval）、生成（Generation）与评判。ICLR 2024。

[15] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, Zhen-Hua Ling. 纠正式检索增强生成。arXiv:2401.15884，2024。

[16] Fangyuan Xu, Weijia Shi, Eunsol Choi. RECOMP：利用上下文压缩（Context Compression）与选择性增强改进检索增强语言模型。ICLR 2024。

[17] Huiqiang Jiang et al. LongLLMLingua：通过提示词压缩，在长上下文场景中加速并增强大语言模型。ACL 2024。

[18] Hung-Ting Chen, Michael Zhang, Eunsol Choi. 丰富的知识

原题名与书目信息：[14] Akari Asai et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024. [15] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, Zhen-Hua Ling. Corrective Retrieval Augmented Generation. arXiv:2401.15884, 2024. [16] Fangyuan Xu, Weijia Shi, Eunsol Choi. RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. ICLR 2024. [17] Huiqiang Jiang et al. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. ACL 2024. [18] Hung-Ting Chen, Michael Zhang, Eunsol Choi. Rich Knowledge